

LAPORAN PRATIKUM PEKAN 7
STRUKTUR DATA
SORTING 2(Shell Sort, Quick Sort dan Merge)



Disusun oleh:
Jose Moniz de Araujo 2511538002

Dosen pengampu:
Wahyudi Dr. S.T.M.T

Asisten Pratikum:
Sherly Sukmadira

DEPARTAMENT INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS
TAHUN
2026

BAB I

PENDAHULUAN

1.1 Latar Belakang

Pengurutan data (Sorting) merupakan proses yang sangat penting dalam ilmu komputer untuk menyusun data ke dalam urutan tertentu agar bisa diolah dan dianalisis. Dalam pemrograman, pengurutan digunakan untuk mempermudah pencarian dan pengelolaan data.

Terdapat berbagai algoritma sorting yang punya fungsi kerja yang berbeda-beda, di antaranya yaitu Shell Sort, Quick Sort dan Merge Sort. Ketiga algoritma digunakan untuk mengurutkan data secara ascending (dari nilai kecil ke nilai yang terbesar). Melalui pratikum ini, mahasiswa dapat bisa pahami cara kerja dan implementasi algoritma shell sort, Quick Sort, dan Merge Sort menggunakan bahasa pemrograman Java.

1.2 Tujuan Pratikum

1. Untuk bisa dapat memahami konsep dasar pada Sorting.
2. Untuk bisa memahami cara kerja algoritma Sorting, Quick Sort dan Merge Sort.
3. Untuk bisa mengimplementasikan algoritma sorting menggunakan bahasa Java.

1.3 manfaat pratikum

1. Menambah pemahaman tentang konsep pengurutan data (sorting).
2. Mengetahui cara kerja shell Sort, Quick sort dan Marge Sort.
3. Melatih kemampuan dalam membuat program menggunakan bahasa Java.

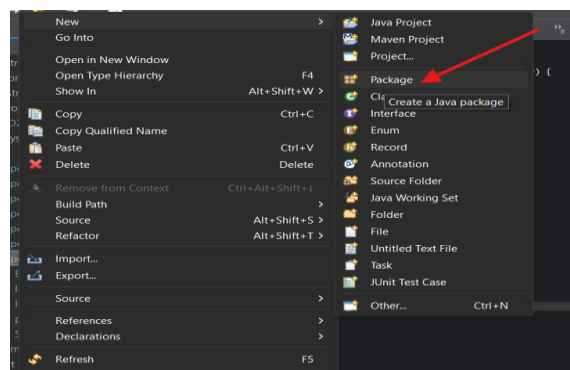
BAB II

PEMBAHASAN

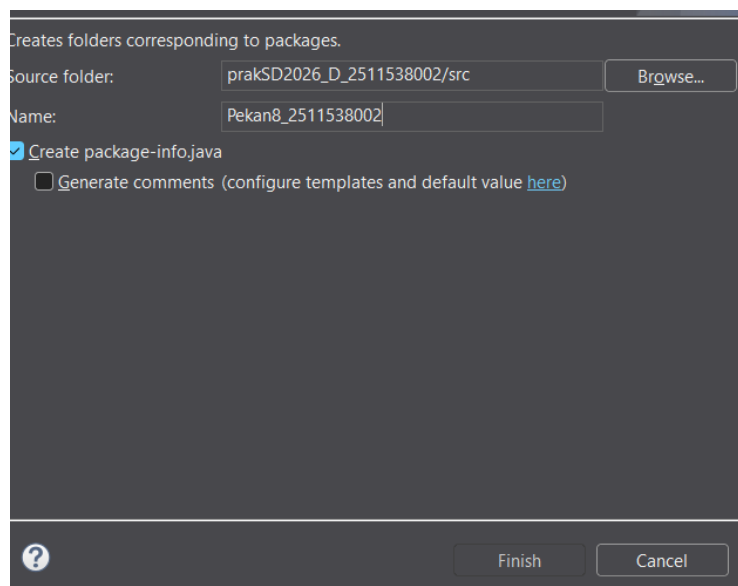
2.2 kode program patikum pekan 8

2.2.1 ShellSort_2511538002

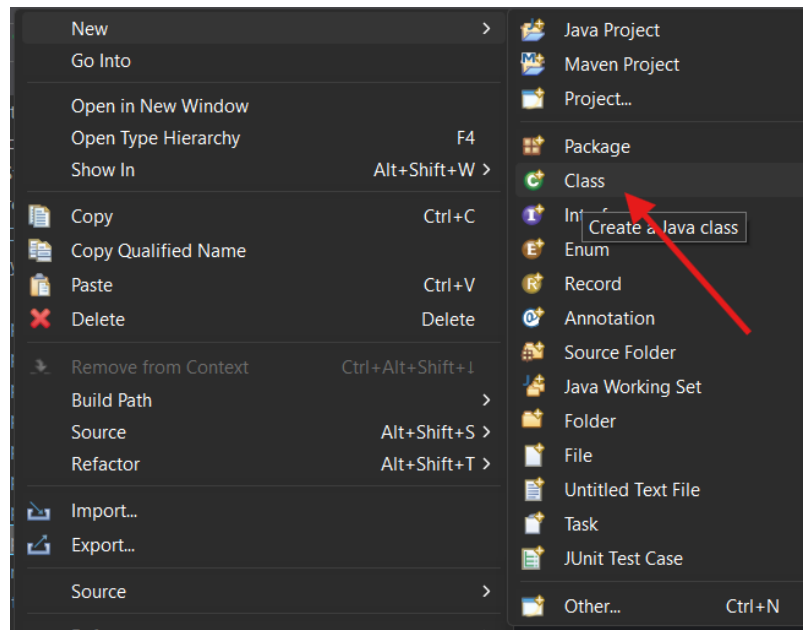
1. Buka aplikasi eclipse untuk membuat package baru pada prakSD2026_D_2511538002 klik kanan, lalu pilih src lalu klik kanan dan pilih “New” dan klik tulisan package.



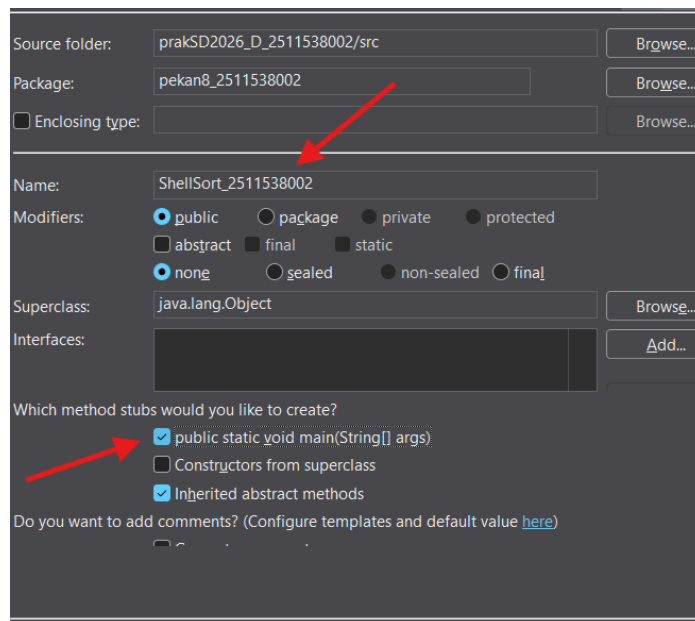
2. Kemudian beri nama pada packagenya dengan “pekan8_2511538002” lalu klik finish



3. Ketika package sudah jadi, kemudian klik kanan pada “pekan8_2511538002” lalu pilih “new” dan pilih “class” untuk membuat kelas baru pada package tersebut.



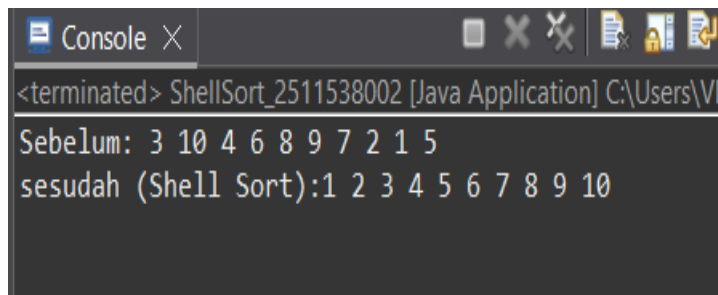
4. Buat nama program yang akan dibuat pada program nama tanpa spasi dan menggunakan huruf kapital pada awal kata, lalu bagian “public static void main(String[] args)”. Kemudian klik “finish”



5. Masukan syntax berikut pada kelas “ShellSort_251138002”

```
1 package pekan8_2511538002;
2
3 public class ShellSort_2511538002 {
4     public static void ShellSort_2511538002(int[] A) {
5         int n = A.length;
6         int gap = n/2;
7         while (gap > 0) {
8             for(int i = gap; i<n; i++) {
9                 int temp = A[i];
10                int j =i;
11                while (j >= gap && A[j - gap] > temp) {
12                    A[j] = A[j- gap];
13                    j = j -gap;
14                }
15                A[j] = temp;
16            }
17            gap = gap/2;
18        }
19    }
20
21
22    public static void main(String[] args) {
23        int [] data = {3, 10, 4, 6, 8, 9, 7, 2, 1, 5};
24
25        System.out.print("Sebelum: ");
26        PrintArray(data);
27
28        ShellSort_2511538002(data);
29
30        System.out.print("sesudah (Shell Sort):");
31        PrintArray(data);
32    }
33
34    public static void PrintArray(int[] arr) {
35        for (int i : arr) System.out.print(i + " ");
36    }
37 }
38
39
40
```

Output dari program tersebut menunjukan data sesudah dan sebelum sangat bedah:



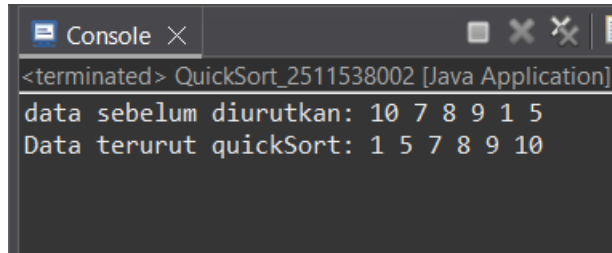
```
<terminated> ShellSort_2511538002 [Java Application] C:\Users\VI
Sebelum: 3 10 4 6 8 9 7 2 1 5
sesudah (Shell Sort):1 2 3 4 5 6 7 8 9 10
```

2.1.2 QuickSort_2511538002

1. Ulangi pembuatan kelas seperti program class sebelumnya dan berikan nama kelas ini sebagai berikut "QuickSort_2511538002" kemudian masukan syntax sebagai berikut:

```
1 package pekan8_2511538002;
2
3 public class QuickSort_2511538002 {
4     static void swap(int[] arr, int i, int j)
5     {
6         int temp = arr[i];
7         arr[i] = arr[j];
8         arr[j] = temp;
9     }
10
11     // metode tambah untuk mengatur pivot menggunakan
12     static void medianOfThree(int[] arr, int low, int high)
13     {
14         int mid = low + (high - low) / 2;
15         // urutkan
16         if (arr[low] > arr[mid]) {
17             swap(arr, low, mid);
18         }
19         if (arr[low] > arr[high]) {
20             swap(arr, low, high);
21         }
22         if (arr[mid] > arr[high]) {
23             swap(arr, mid, high);
24         }
25         swap(arr, mid, high);
26     }
27     static int partition (int[] arr, int low, int high)
28     {
29         // panggil
30         medianOfThree(arr, low, high);
31         int pivot = arr[high]; // sekanang
32         int i = (low - 1);
33         for (int j = low; j <= high - 1; j++) {
34             // jika
35             if (arr[j] < pivot) {
36                 // increment
37                 i++;
38                 swap(arr, i, j);
39             }
40         }
41         swap(arr, i + 1, high);
42         return (i + 1);
43     }
44     static void quickSort(int[] arr, int low, int high)
45     {
46         if (low < high) {
47             int pi = partition(arr, low, high);
48             quickSort(arr, low, pi - 1);
49             quickSort(arr, pi + 1, high);
50         }
51     }
52 }
53
54 public static void PrintArr(int[] arr)
55 {
56     for (int i = 0; i < arr.length; i++) {
57         System.out.print(arr[i] + " ");
58     }
59     System.out.println();
60 }
61
62 public static void main(String[] args) {
63     {
64         int[] arr = { 10, 7, 8, 9, 1, 5 };
65         int n = arr.length;
66         System.out.print("data sebelum diurutkan: ");
67         PrintArr(arr);
68         quickSort(arr, 0, n - 1);
69         System.out.print("Data terurut quickSort_2511538002: ");
70         PrintArr(arr);
71     }
72 }
73 }
```

Kemudian ini hasil dari program tersebut



```
<terminated> QuickSort_2511538002 [Java Application]
data sebelum diurutkan: 10 7 8 9 1 5
Data terurut quickSort: 1 5 7 8 9 10
```

2.1.3 MargeSort_2511538002

1. Ulangi pembuatan kelas seperti program class sebelumnya dan berikan nama kelas ini sebagai berikut “MargeSort_2511638002” kemudian masukan syntax sebagai berikut:

```
1 package pekan8_2511538002;
2
3 public class MargeSort_2511538002 {
4     void merge(int arr[], int l, int m, int r) {
5         // Ukuran array sementara
6         int n1 = m - l + 1;
7         int n2 = r - m;
8         int L[] = new int[n1];
9         int R[] = new int[n2];
10        // Salin data ke array sementara L[]
11        for (int i = 0; i < n1; i++) {
12            L[i] = arr[l + i];
13        }
14        // Salin data ke array sementara R[]
15        for (int j = 0; j < n2; j++) {
16            R[j] = arr[m + 1 + j];
17        }
18        int i = 0;
19        int j = 0;
20        int k = l;
21        // Gabungkan kedua array sementara
22        while (i < n1 && j < n2) {
23            if (L[i] <= R[j]) {
24                arr[k] = L[i];
25                i++;
26            } else {
27                arr[k] = R[j];
28                j++;
29            }
30            k++;
31        }
32        // Salin sisa elemen L[]
33        while (i < n1) {
34            arr[k] = L[i];
35            i++;
36            k++;
37        }
38        // Salin sisa elemen R[]
39        while (j < n2) {
40            arr[k] = R[j];
41            j++;
42            k++;
43        }
44    }
45 }
```

```

43     }
44 }
45 // Fungsi Merge Sort
46 void sort(int arr[], int l, int r) {
47     if (l < r) {
48         int m = (l + r) / 2;
49         // Ucutkan bagian kiri
50         sort(arr, l, m);
51         // Ucutkan bagian kanan
52         sort(arr, m + 1, r);
53         // Gabungkan hasilnya
54         merge(arr, l, m, r);
55     }
56 }
57 // Menampilkan array
58 static void PrintArray(int arr[]) {
59     for (int i = 0; i < arr.length; i++) {
60         System.out.print(arr[i] + " ");
61     }
62     System.out.println();
63 }
64 }
65
66 public static void main(String[] args) {
67     int arr[] = {12, 11, 13, 5, 6, 7};
68     System.out.println("Sebelum diurutkan:");
69     PrintArray(arr);
70     MergeSort_2511538002 ob = new MergeSort_2511538002();
71     ob.sort(arr, 0, arr.length - 1);
72     System.out.println("\nSesudah diurutkan menggunakan Merge Sort:");
73     PrintArray(arr);
74 }
75 }

```

```

<terminated> MergeSort_2511538002 [Java Application] C:\Users\
Sebelum diurutkan:
12 11 13 5 6 7

Sesudah diurutkan menggunakan Merge Sort:
5 6 7 11 12 13

```

2.1.3 BubleSort_2511538002

1. Ulangi pembuatan kelas seperti program class sebelumnya dan berikan nama kelas ini sebagai berikut “BubleSort_2511638002” kemudian masukan syntax sebagai berikut:

```

1 package pekan8_2511538002;
2
3 import java.awt.BorderLayout;
4
5 public class BubleSortGUI_2511538002 extends JFrame {
6     private static final long serialVersionUID = 1L;
7     private int[] array_8002;
8     private JLabel[] labelArray_8002;
9     private JButton stepButton_8002, resetButton_8002, setButton_8002;
10    private JTextField inputField_8002;
11    private JPanel panelArray_8002;
12    private JTextArea stepArea_8002;
13    private int i_8002 = 0, j_8002 = 0;
14    private boolean sorting_8002 = false;
15    private int stepCount_8002 = 1;
16    public BubleSortGUI_2511538002() {
17        setTitle("Buble Sort Langkah per Langkah");
18        setSize(750, 400);
19        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
20        setLocationRelativeTo(null);
21        setLayout(new BorderLayout());
22
23        // panel input
24        JPanel inputPanel_8002 = new JPanel(new FlowLayout());
25        inputField_8002 = new JTextField(30);
26        setButton_8002 = new JButton("Set Array");
27        inputPanel_8002.add(new JLabel("Masukan angka (pisahkan dengan koma)"));
28        inputPanel_8002.add(inputField_8002);
29        inputPanel_8002.add(setButton_8002);
30
31        // panel array visual
32        panelArray_8002 = new JPanel();
33        panelArray_8002.setLayout(new FlowLayout());
34
35        // panel kontrol
36        JPanel controlPanel_8002 = new JPanel();
37        stepButton_8002 = new JButton("Langkah selanjutnya");
38        resetButton_8002 = new JButton("Reset");
39        controlPanel_8002.add(stepButton_8002);
40        controlPanel_8002.add(resetButton_8002);
41    }
42 }

```



```

57
58 // area text untuk log langkah langkah
59 stepArea_8002 = new JTextArea(8, 60);
60 stepArea_8002.setEditable(false);
61 stepArea_8002.setFont(new Font("Monospaced", Font.PLAIN, 14));
62 JScrollPane scrollPane_8002 = new JScrollPane(stepArea_8002);
63
64 // tambahkan panel ke frame
65 add(inputPanel_8002, BorderLayout.NORTH);
66 add(panelArray_8002, BorderLayout.CENTER);
67 add(controlPanel_8002, BorderLayout.SOUTH);
68 add(scrollPane_8002, BorderLayout.EAST);
69
70 // event set array
71 setButton_8002.addActionListener(e -> setArrayFromInput_8002());
72
73 // event selanjutnya
74 stepButton_8002.addActionListener(e -> performStep_8002());
75
76 // event reset
77 resetButton_8002.addActionListener(e -> reset_8002());
78 }
79
80 private void setArrayFromInput_8002() {
81     String text_8002 = inputField_8002.getText().trim();
82     if (text_8002.isEmpty()) return;
83     String[] parts_8002 = text_8002.split(",");
84     array_8002 = new int[parts_8002.length];
85     try {
86         for (int k = 0; k < parts_8002.length; k++) {
87             array_8002[k] = Integer.parseInt(parts_8002[k].trim());
88         }
89     } catch (NumberFormatException e) {
90         JOptionPane.showMessageDialog(this, "Masukkan hanya angka "
91             + "yang dipisahkan koma!", "Error", JOptionPane.ERROR_MESSAGE);
92     }
93     return;
94 }

```

```

94
95 i_8002 = 0;
96 j_8002 = 0;
97 stepCount_8002 = 1;
98 sorting_8002 = true;
99 stepButton_8002.setEnabled(true);
100 stepArea_8002.setText("");
101 panelArray_8002.removeAll();
102 labelArray_8002 = new JLabel[array_8002.length];
103 for (int k = 0; k < array_8002.length; k++) {
104     labelArray_8002[k] = new JLabel(String.valueOf(array_8002[k]));
105     labelArray_8002[k].setFont(new Font("Arial", Font.BOLD, 24));
106     labelArray_8002[k].setOpaque(true);
107     labelArray_8002[k].setBackground(Color.WHITE);
108     labelArray_8002[k].setBorder(BorderFactory.createLineBorder(Color.BLACK));
109     labelArray_8002[k].setPreferredSize(new Dimension(50, 50));
110     labelArray_8002[k].setHorizontalAlignment(SwingConstants.CENTER);
111     panelArray_8002.add(labelArray_8002[k]);
112 }
113
114 panelArray_8002.revalidate();
115 panelArray_8002.repaint();
116 }
117
118 private void performStep_8002() {
119     if (!sorting_8002 || i_8002 >= array_8002.length - 1) {
120         sorting_8002 = false;
121         stepButton_8002.setEnabled(false);
122         JOptionPane.showMessageDialog(this, "Sorting selesai!");
123         return;
124     }
125
126     resetHighlights_8002();
127     StringBuilder stepLog_8002 = new StringBuilder();
128
129     labelArray_8002[j_8002].setBackground(Color.CYAN);
130     labelArray_8002[j_8002 + 1].setBackground(Color.CYAN);
131
132     if (array_8002[j_8002] > array_8002[j_8002 + 1]) {
133         // Swap

```

```

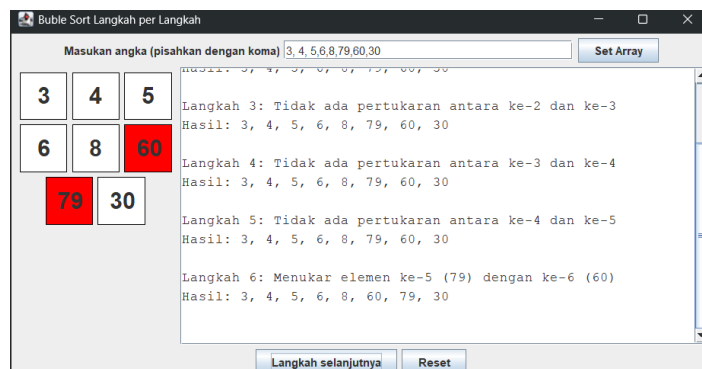
134         int temp_8002 = array_8002[j_8002];
135         array_8002[j_8002] = array_8002[j_8002 + 1];
136         array_8002[j_8002 + 1] = temp_8002;
137         labelArray_8002[j_8002].setBackground(Color.RED);
138         labelArray_8002[j_8002 + 1].setBackground(Color.RED);
139         stepLog_8002.append("Langkah ").append(stepCount_8002).append(": Menukar elemen ke-")
140             .append(j_8002).append(" ").append(array_8002[j_8002 + 1]).append(" dengan ke-")
141             .append(j_8002 + 1).append(" ").append(array_8002[j_8002]).append("\n");
142     } else {
143         stepLog_8002.append("Langkah ").append(stepCount_8002).append(": Tidak ada pertukaran antara ke-")
144             .append(j_8002).append(" dan ke-").append(j_8002 + 1).append("\n");
145     }
146
147     stepLog_8002.append("Hasil: ").append(arrayToString_8002(array_8002)).append("\n\n");
148     stepArea_8002.append(stepLog_8002.toString());
149     updateLabels_8002();
150
151     j_8002++;
152     if (j_8002 >= array_8002.length - i_8002 - 1) {
153         j_8002 = 0;
154         i_8002++;
155     }
156     stepCount_8002++;
157
158     if (i_8002 >= array_8002.length - 1) {
159         sorting_8002 = false;
160         stepButton_8002.setEnabled(false);
161         JOptionPane.showMessageDialog(this, "Sorting selesai!");
162     }
163 }
164
165 private void updateLabels_8002() {
166     for (int k = 0; k < array_8002.length; k++) {
167         labelArray_8002[k].setText(String.valueOf(array_8002[k]));
168     }
169 }
170

```

```

170
171 private void resetHighlights_8002() {
172     for (JLabel label : labelArray_8002) {
173         label.setBackground(Color.WHITE);
174     }
175 }
176
177 private void reset_8002() {
178     inputField_8002.setText("");
179     panelArray_8002.removeAll();
180     panelArray_8002.revalidate();
181     panelArray_8002.repaint();
182     stepArea_8002.setText("");
183     stepButton_8002.setEnabled(false);
184     sorting_8002 = false;
185     i_8002 = 0;
186     j_8002 = 0;
187     stepCount_8002 = 1;
188 }
189
190 private String arrayToString_8002(int[] arr) {
191     StringBuilder sb_8002 = new StringBuilder();
192     for (int k = 0; k < arr.length; k++) {
193         sb_8002.append(arr[k]);
194         if (k < arr.length - 1) sb_8002.append(", ");
195     }
196     return sb_8002.toString();
197 }
198
199 public static void main(String[] args) {
200     SwingUtilities.invokeLater(() -> {
201         BubbleSortGUI_2511538002 gui = new BubbleSortGUI_2511538002();
202         gui.setVisible(true);
203     });
204 }
205 }

```



2.1.3 MargeSortGUI_2511538002

1. Ulangi pembuatan kelas seperti program class sebelumnya dan berikan nama kelas nia sebagai beriku “MargeSortGUI_2511638002” kemdian masukan syntax sebagai berikut:

```
1 package pekan8_2511538002;
2
3 import java.awt.BorderLayout;
4
23 public class MergeSortGUI_2511538002 extends JFrame {
24
25     private static final long serialVersionUID = 1L;
26     private int[] array_8002;
27     private JLabel[] labelArray_8002;
28     private JButton stepButton_8002, resetButton_8002, setButton_8002;
29     private JTextField inputField_8002;
30     private JPanel panelArray_8002;
31     private JTextArea stepArea_8002;
32     private Queue<int[]> mergeQueue_8002 = new LinkedList<>();
33     private int stepCount_8002 = 1;
34     private boolean isMerging_8002 = false;
35     private boolean copying_8002 = false;
36
37     private int[] temp_8002;
38     private int left_8002, mid_8002, right_8002;
39     private int i_8002, j_8002, k_8002;
40
41
42     public MergeSortGUI_2511538002() {
43         setTitle("Merge Sort Langkah per Langkah");
44         setSize(750, 400);
45         setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
46         setLocationRelativeTo(null);
47         setLayout(new BorderLayout());
48
49         // panel input
50         JPanel inputPanel_8002 = new JPanel(new FlowLayout());
51         inputField_8002 = new JTextField(30);
52         setButton_8002 = new JButton("Set Array");
53         inputPanel_8002.add(new JLabel("Masukan angka (pisahkan dengan koma)"));
54         inputPanel_8002.add(inputField_8002);
55
56
57         // panel array visual
58         panelArray_8002 = new JPanel();
59         panelArray_8002.setLayout(new FlowLayout());
60
61         // panel kontrol
62         JPanel controlPanel_8002 = new JPanel();
63         stepButton_8002 = new JButton("Langkah selanjutnya");
64         resetButton_8002 = new JButton("Reset");
65         controlPanel_8002.add(stepButton_8002);
66         controlPanel_8002.add(resetButton_8002);
67
68         // area text untuk log langkah langkah
69         stepArea_8002 = new JTextArea(8, 60);
70         stepArea_8002.setEditable(false);
71         stepArea_8002.setFont(new Font("Monospaced", Font.PLAIN, 14));
72         JScrollPane scrollPane_8002 = new JScrollPane(stepArea_8002);
73
74         // tambahkan panel ke frame
75         add(inputPanel_8002, BorderLayout.NORTH);
76         add(panelArray_8002, BorderLayout.CENTER);
77         add(controlPanel_8002, BorderLayout.SOUTH);
78         add(scrollPane_8002, BorderLayout.EAST);
79
80         // event set array
81         setButton_8002.addActionListener(e -> setArrayFromInput_8002());
82
83         // event selanjutnya
84         stepButton_8002.addActionListener(e -> performStep_8002());
85
86         // event reset
87         resetButton_8002.addActionListener(e -> reset_8002());
88     }
89 }
```

```

90 private void setArrayFromInput_8002() {
91     String text_8002 = inputField_8002.getText().trim();
92     if (text_8002.isEmpty()) return;
93     String[] parts_8002 = text_8002.split(",");
94     array_8002 = new int[parts_8002.length];
95     try {
96         for (int k = 0; k < parts_8002.length; k++) {
97             array_8002[k] = Integer.parseInt(parts_8002[k].trim());
98         }
99     } catch (NumberFormatException e) {
100         JOptionPane.showMessageDialog(this, "Masukkan hanya angka dengan koma!",
101             "Error", JOptionPane.ERROR_MESSAGE);
102         return;
103     }
104
105     labelArray_8002 = new JLabel[array_8002.length];
106     panelArray_8002.removeAll();
107     for (int k = 0; k < array_8002.length; k++) {
108         labelArray_8002[k] = new JLabel(String.valueOf(array_8002[k]));
109         labelArray_8002[k].setFont(new Font("Arial", Font.BOLD, 24));
110         labelArray_8002[k].setOpaque(true);
111         labelArray_8002[k].setBackground(Color.WHITE);
112         labelArray_8002[k].setBorder(BorderFactory.createLineBorder(Color.BLACK));
113         labelArray_8002[k].setPreferredSize(new Dimension(50, 50));
114         labelArray_8002[k].setHorizontalAlignment(SwingConstants.CENTER);
115         panelArray_8002.add(labelArray_8002[k]);
116     }
117
118     mergeQueue_8002.clear();
119     generateMergeSteps_8002(0, array_8002.length - 1);
120     stepButton_8002.setEnabled(true);
121     stepArea_8002.setText("");
122     stepCount_8002 = 1;
123     isMerging_8002 = false;
124
125     panelArray_8002.revalidate();
126     panelArray_8002.repaint();
127 }

```

```

128 private void generateMergeSteps_8002(int left, int right) {
129     if (left >= right) return;
130     int mid = (left + right) / 2;
131     generateMergeSteps_8002(left, mid);
132     generateMergeSteps_8002(mid + 1, right);
133     mergeQueue_8002.add(new int[]{left, mid, right});
134 }
135
136 private void performStep_8002() {
137     resetHighlights_8002();
138
139     if (!isMerging_8002 && !mergeQueue_8002.isEmpty()) {
140         int[] range_8002 = mergeQueue_8002.poll();
141         left_8002 = range_8002[0];
142         mid_8002 = range_8002[1];
143         right_8002 = range_8002[2];
144         temp_8002 = new int[right_8002 - left_8002 + 1];
145         i_8002 = left_8002;
146         j_8002 = mid_8002 + 1;
147         k_8002 = 0;
148         copying_8002 = false;
149         isMerging_8002 = true;
150         stepArea_8002.append("Langkah " + stepCount_8002++ +
151             ": Mulai merge dari " + left_8002 + " ke " + right_8002 + "\n");
152         return;
153     }
154
155     if (isMerging_8002 && !copying_8002) {
156         if (i_8002 <= mid_8002 && j_8002 <= right_8002) {
157             labelArray_8002[i_8002].setBackground(Color.CYAN);
158             labelArray_8002[j_8002].setBackground(Color.CYAN);
159             if (array_8002[i_8002] <= array_8002[j_8002]) {
160                 temp_8002[k_8002++] = array_8002[i_8002++];
161             } else {
162                 temp_8002[k_8002++] = array_8002[j_8002++];
163             }
164         }
165         stepArea_8002.append("Langkah " + stepCount_8002++ + ": Bandingkan dan salin elemen\n");
166         return;

```

```

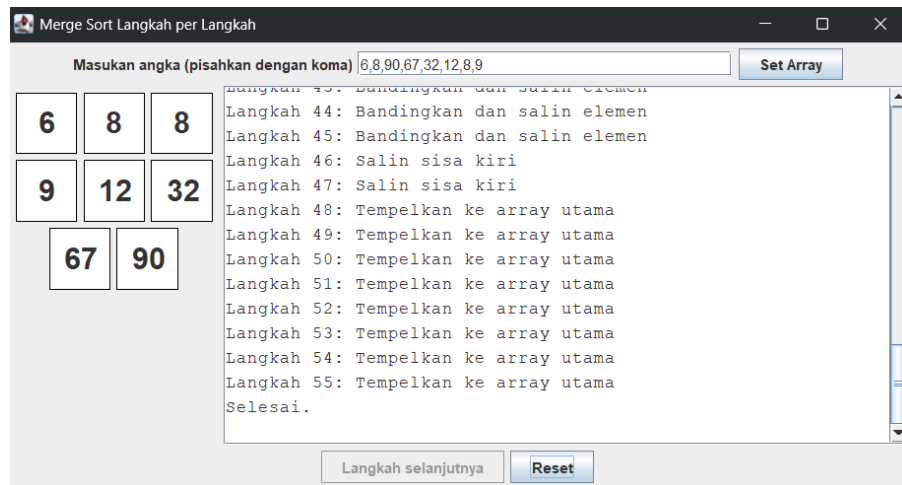
167     } else if (i_8002 <= mid_8002) {
168         temp_8002[k_8002++] = array_8002[i_8002++];
169         stepArea_8002.append("Langkah " + stepCount_8002++ + ": Salin sisa kiri\n");
170         return;
171     } else if (j_8002 <= right_8002) {
172         temp_8002[k_8002++] = array_8002[j_8002++];
173         stepArea_8002.append("Langkah " + stepCount_8002++ + ": Salin sisa kanan\n");
174         return;
175     } else {
176         copying_8002 = true;
177         k_8002 = 0;
178         return;
179     }
180 }
181
182 if (copying_8002 && k_8002 < temp_8002.length) {
183     array_8002[left_8002 + k_8002] = temp_8002[k_8002];
184     labelArray_8002[left_8002 + k_8002].setText(String.valueOf(temp_8002[k_8002]));
185     labelArray_8002[left_8002 + k_8002].setBackground(Color.GREEN);
186     k_8002++;
187     stepArea_8002.append("Langkah " + stepCount_8002++ + ": Tempelkan ke array utama\n");
188     return;
189 }
190
191 if (copying_8002 && k_8002 == temp_8002.length) {
192     isMerging_8002 = false;
193     copying_8002 = false;
194 }
195
196 if (mergeQueue_8002.isEmpty() && !isMerging_8002) {
197     stepArea_8002.append("Selesai.\n");
198     stepButton_8002.setEnabled(false);
199     JOptionPane.showMessageDialog(this, "Merge Sort selesai!");
200 }
201 }
202
203 private void resetHighlights_8002() {
204     if (labelArray_8002 == null) return;
205     for (JLabel label : labelArray_8002) {
206         label.setBackground(Color.WHITE);

```

```

207     }
208 }
209
210 private void reset_8002() {
211     inputField_8002.setText("");
212     panelArray_8002.removeAll();
213     panelArray_8002.revalidate();
214     panelArray_8002.repaint();
215     stepArea_8002.setText("");
216     stepButton_8002.setEnabled(false);
217     mergeQueue_8002.clear();
218     isMerging_8002 = false;
219     stepCount_8002 = 1;
220 }
221
222 public static void main(String[] args) {
223     SwingUtilities.invokeLater(() -> {
224         MergeSortGUI_2511538002 gui = new MergeSortGUI_2511538002();
225         gui.setVisible(true);
226     });
227 }
228 }

```



2.2 Tugas pratikum pekan 8

2.2.1 Soal Tugas

1. Mahasiswa diminta mengimplementasikan salah SATU algoritma sorting untuk mengurutkan data lagu dalam playlist. Pilih salah satu: Shell Sort, Quick Sort, atau Merge Sort. Data lagu disimpan dalam array (bukan linked list). Program harus bisa mengurutkan berdasarkan Judul (AZ) atau Durasi (terpendek ke terpanjang).

2.2.2 Kode program Lagu_2511538002

```

1 package pekan8_2511538002;
2
3 public class Lagu_2511538002 {
4     String judul_8002;
5     String penyanyi_8002;
6     int durasi_8002;
7
8     // constructor untuk mengisi data lagu saat objek dibuat
9     public Lagu_2511538002(String judul_8002, String penyanyi_8002, int durasi_8002) {
10         this.judul_8002 = judul_8002;
11         this.penyanyi_8002 = penyanyi_8002;
12         this.durasi_8002 = durasi_8002;
13     }
14 }
15
16

```

2.2.3 Penejelasan kode program Lagu_2511538002

1. Deklarasi kelas

```
public class Lagu_2511538002 {
```

Kelas Lagu_2511538002 berfungsi sebagai objek lagu yang akan digunakan dalam program ini.

2. Deklarasi Atribut

```
String judul_8002;  
String penyanyi_8002;  
int durasi_8002;
```

Atribut digunakan supaya bisa menyimpan data dari setiap lagu seperti judul lagu, nama penyanyi, dan untuk bisa menyimpan durasi lagu dalam satuan detik.

3. Constructor

```
public Lagu_2511538002(String judul_8002,  
String penyanyi_8002,  
int durasi_8002) {
```

Constructor adalah method khusus yang dipanggil secara otomatis ketika objek dibuat. Constructor digunakan untuk menginisialisasi atau memberikan nilai awal pada atribut objek.

4. Pengguna keyword this

```
this.judul_8002 = judul_8002;  
this.penyanyi_8002 = penyanyi_8002;  
this.durasi_8002 = durasi_8002;
```

Keyword this digunakan untuk merujuk pada atribut yang dimiliki oleh objek saat ini

2.2.4 kode program Sorting_2511538002

```
1 package pekan8_2511538002;
2 public class Sorting_2511538002 {
3
4     Lagu_2511538002[] datalagu_8002 = new Lagu_2511538002[20];
5     int jumlahData_8002 = 0;
6     public void inputData_8002() {
7         datalagu_8002[jumlahData_8002++] =
8             new Lagu_2511538002("Mio Cristo painge Diamanti", "penyanyi A", 270);
9         datalagu_8002[jumlahData_8002++] =
10             new Lagu_2511538002("La Rumba Del perdon", "penany B", 252);
11         datalagu_8002[jumlahData_8002++] =
12             new Lagu_2511538002("La Perla", "penyanyi C", 196);
13
14         datalagu_8002[jumlahData_8002++] =
15             new Lagu_2511538002("perfect", "Ed Sheeran", 263);
16
17         datalagu_8002[jumlahData_8002++] =
18             new Lagu_2511538002("photograh", "Ed Sheeran", 250);
19
20         datalagu_8002[jumlahData_8002++] =
21             new Lagu_2511538002("Memories", "Maroon 5", 189);
22
23         datalagu_8002[jumlahData_8002++] =
24             new Lagu_2511538002("counting Stars", "OneRepublic", 257);
25     }
26     //method untuk menampilkan seluruh data lagu
27
28     public void tampilData_8002() {
29
30         for (int i = 0; i < jumlahData_8002; i++) {
31
32             System.out.println(
33                 (i+1)+ " . "
34                 + datalagu_8002[i].judul_8002
35                 + " . "
36                 + datalagu_8002[i].durasi_8002
37                 + " detik");
38         }
39     }
40
41     // method utama quick Sort
42     public void quickSort_8002(int low, int high) {
43         if (low < high) {
44             int pi = partition_8002(low, high);
45             quickSort_8002(low, pi - 1);
46             quickSort_8002(pi + 1, high);
47         }
48     }
49     public int partition_8002(int low, int high) {
50         int pivot = datalagu_8002[high].durasi_8002;
51         int i = low - 1;
52         for (int j = low; j < high; j++) {
53             if (datalagu_8002[j].durasi_8002 < pivot) {
54
55                 i++;
56
57                 // tukar data
58                 Lagu_2511538002 temp = datalagu_8002[i + 1];
59                 datalagu_8002[i + 1] = datalagu_8002[high];
60                 datalagu_8002[high] = temp;
61
62             }
63         }
64         //menempatkan pivot ke posisi yang benar
65         Lagu_2511538002 temp = datalagu_8002[i + 1];
66         datalagu_8002[i + 1] = datalagu_8002[high];
67         datalagu_8002[high] = temp;
68
69         // mengembalikan index pivot
70         return i + 1;
71     }
72
73     // program utama
74
75     public static void main(String[] args) {
76
77         // membuat objek playlist
78         Sorting_2511538002 playlist_8002 =
79             new Sorting_2511538002();
80
81         // mengisi data lagu
82         playlist_8002.inputData_8002();
83         System.out.println("=== Sorting Playlist NIM: 2511538002 ===");
84         System.out.println("Pilih Algoritma (1=Shell, 2=Quick, 3=Merge): 2");
85
86         System.out.println("\n=== Data Sebelum Sorting ===");
87         playlist_8002.tampilData_8002();
88
89         // menjalankan quick Sort
90         playlist_8002.quickSort_8002(
91             0, playlist_8002.jumlahData_8002 - 1);
92
93         //menampilkan data setelah sorting
94         System.out.println("\n data setelah quick Sort (durasi Ascending) ===");
95         playlist_8002.tampilData_8002();
96     }
97 }
98
99
```


2.2.5 Penejelasan kode program Lagu_2511538002

1. Deklarasi kelas

```
public class Sorting_2511538002 {
```

Kelas Sorting merupakan keals utama yang digunakan untuk mengelola data palylist lagu dan menjalangkan algoritma quick Sort, dan menggunakan array untuk menyimpan data lagu, method untuk memasukan data, menampilkan data, melakukan soting dan method utama.

2. Deklarasi Array dan Variabel

```
Lagu_2511538002[] datalagu_8002 = new Lagu_2511538002[20];  
int jumlahData_8002 = 0;
```

Program ini di baris pertama digunakan untuk melakukan array yang dapat menyimpan maksimal 20 objek lagu.

3. Method Input Data

```
public void inputData_8002()
```

Method ini digunakan untuk bsia mengisi data lag uke dalam Array, data yang dimsaunk terdiri dari atribut sebagai judul lagu, nama penyani dan durasi lagu.

4. Method manmpilkan data

```
public void tampilData_8002()  
for (int i = 0; i < jumlahData_8002; i++)
```

Method ini digunakan untuk menampilkan seluruh data lagu yang terdapat dalam array, dan perulangan untuk bisa membaca dari indeks pertama hingga indeks terakhir.

5. Method Quick Sort

```
public void quickSort_8002(int low, int high)
if (low < high)
```

Method ini merupakan implementasi algoritma Quick Sort bekerja dengan cara memilih pivot membagi data menjadi dua bagian dan mengurutkan bagian kiri dan kanan secara rekursif.

6. Method Partiton

```
public int partition_8002(int low, int high)
int pivot = dataLagu_8002[high].durasi_8002;
return i + 1;
```

Method ini digunakan untuk menentukan posisi pivot yang benar dan dipilih dari elemen yang terakhir. Kemudian setiap data dibandingkan dengan pivot.

7. Method Main

```
public static void main(String[] args)
Sorting_2511538002 playlist_8002 =
    new Sorting_2511538002();
playlist_8002.inputData_8002();
System.out.println("=== Sorting Playlist NIM: 2511538002 ===");
playlist_8002.tampilData_8002();
    playlist_8002.quickSort_8002( 0,
        playlist_8002.jumlahData_8002 - 1
    );
playlist_8002.tampilData_8002();
```

Method main merupakan titik awal eksekusi program yang dilakukan sebagai berikut:

- Sorting_2511538002 di baris kedua digunakan untuk membuat objek yang akan menjalankan seluruh method dalam program.
- Playlist digunakan untuk bisa mengisi data ke dalam array

- System.out.println digunakan untuk bisa menampilkan identitas program.
- Playlist digunakan untuk menjalankan Quick Sort untuk mengurutkan data berdasarkan durasi yang terkecil ke terbesar.
- Playlist yang terakhir untuk bisa menampilkan data setelah selesai diurutkan

Dari program Sorting_2511538002 hasil outputnya sebagai berikut:

```

=== Sorting Playlist NIM: 2511538002 ===
Pilih Algoritma (1=Shell, 2=Quick, 3=Merge): 2

=== Data Sebelum Sorting ===
1. Mio Cristo painge Diamanti - 270 detik
2. La Rumba Del perdón - 252 detik
3. La Perla - 196 detik
4. perfect - 263 detik
5. photograh - 258 detik
6. Memories - 189 detik
7. counting Stars - 257 detik

=== data setelah quick Sort (durasi Ascending) ===
1. La Rumba Del perdón - 252 detik
2. counting Stars - 257 detik
3. Mio Cristo painge Diamanti - 270 detik
4. perfect - 263 detik
5. photograh - 258 detik
6. Memories - 189 detik
7. La Perla - 196 detik

```

Tujuan saya memilih Quick Sort:

Di tugas ini saya memilih Quick Sort karena memiliki kecepatan yang tinggi, penggunaan memori yang sangat relatif yang kecil, dan cocok mengurutkan data playlist lagu berdasarkan durasi, dibandingkan dengan Shell Sort, Quick Sort lebih efisien. Sementara dibandingkan Merge Sort, Quick Sort lebih hemat memori dan lebih sederhana untuk diterapkan pada tugas ini.

BAB III

PENUTUP

3.1 Kesimpulan

Setelah praktikum ini, konsep algoritma yang berhasil saya terapkan dalam bahasa pemrograman Java menjadi lebih jelas. Dalam praktikum ini, saya mempelajari algoritma pengurutan seperti Bubble Sort, Shell Sort, Merge Sort, dan Quick Sort. Saya juga mengingat kembali algoritma-algoritma awal dalam program konsol dan GUI. Setiap algoritma memiliki cara kerja yang unik, lebih dan kurang. Misalnya, Bubble Sort cukup mudah dipahami, tetapi tidak terlalu efisien jika digunakan untuk mengurutkan kubus yang sangat besar. Shell Sort kemudian menjadi yang terpenting dibandingkan Bubble Sort atau Insertion Sort, sehingga prosesnya lebih baik. Merge Sort memiliki kinerja yang baik dan stabil, tetapi membutuhkan memori tambahan. Quick Sort kemudian menjadi algoritma yang sangat presisi dan mudah digunakan untuk mengurutkan dadu dalam array.

DAFTAR PUSTAKA

Abdul Kadir. 2020. *Pemrograman Java untuk Pemula*. Yogyakarta: Andi Publisher.

Rosa A. S. dan M. Shalahuddin. 2018. *Rekayasa Perangkat Lunak Terstruktur dan Berorientasi Objek*. Bandung: Informatika.

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, dan Clifford Stein. 2022. *Introduction to Algorithms*. Cambridge: MIT Press.